

# SAS Alert Engine — Onboarding & Build Specification

Everything a new engineer needs to build the milestone alert engine from zero. From the milestone-module team (Ronaka) · 2026-08-16

**Read this first.** You are inheriting the **alert engine** — the component that decides *when to email whom* about shipment milestones. The rest of the system (data sync, the milestone builder, templates, assignment) is already built and produces well-defined data for you. This document explains the whole system briefly, then specifies exactly what your engine must read, evaluate, and do. You do not need to change any other module.

## 1. What is SAS, in one minute

SAS (Shipment Alerting System) sits on top of Dart Global Logistics' CargoWise freight data. A background **sync** pulls shipment records from the CargoWise API into a Supabase/PostgreSQL database. Admins define **milestones** (checkpoints like 'cargo picked up', 'documents uploaded') in a visual **builder**, group them into **templates**, and **assign** a template to shipments. Each assignment creates rows the engine watches. Your **alert engine** evaluates those rows on a schedule and sends alert emails when a milestone is overdue or a condition is met.

Stack: Next.js frontend, Python Flask backend, Supabase (PostgreSQL). The engine is Python on the backend, run on a scheduler (APScheduler) like the other jobs.

## 2. The data flow (where you sit)

```
CargoWise API
| (background sync, hourly)
v
shipments ..... raw_json (full payload) + milestones (jsonb, curated)
^ |
| milestone_field_map tells | the builder's field references
| the sync which fields to pull | resolve against this jsonb
|
Admin -> Milestone Builder -> milestone_library -> Template -> ASSIGN
|
v
shipment_milestones (one row per
milestone per shipment; frozen snapshot)
|
v
>>> YOUR ALERT ENGINE reads these,
evaluates, sends emails, updates status
```

You only ever **read** `shipment_milestones` (and the shipment it points to). You never write to `shipments.milestones` — only the sync does that.

## 3. What the engine is responsible for

- On a schedule, load every active (not-completed) `shipment_milestones` row.
- For each, decide whether the milestone is **satisfied** or **outstanding** by evaluating its check(s) against the shipment's live data.
- For each alert rule on the milestone, decide if it should fire *now* (timing + fire condition + recurrence + stop condition).
- Resolve the recipient and send the email.

■ Update `shipment_milestones.status` (pending / current / overdue / completed) and set due dates for 'after previous milestone' milestones.

## 4. The row you read: `shipment_milestones`

Each assigned milestone is one row. The important columns:

| Column                            | Meaning                                                                                           |
|-----------------------------------|---------------------------------------------------------------------------------------------------|
| <code>id</code>                   | Primary key of this assigned milestone.                                                           |
| <code>shipment_id</code>          | FK to shipments. Load this shipment to get field values.                                          |
| <code>sequence_order</code>       | 0-based position within the template (0,1,2...). Needed for 'after previous milestone' due dates. |
| <code>status</code>               | 'pending'   'current'   'overdue'   'completed'. You maintain this.                               |
| <code>due_date</code>             | The deadline (ISO string) or null. Rule timing is measured from here.                             |
| <code>is_critical</code>          | Bool. Critical milestones are highlighted; you may escalate.                                      |
| <code>milestone_type</code>       | date   missing   comparison   document   status   custom.                                         |
| <code>milestone_snapshot</code>   | jsonb — the full milestone config, frozen at assignment (section 6).                              |
| <code>alert_rules_snapshot</code> | jsonb array — the alert rules, frozen at assignment (section 10).                                 |

**Why 'snapshot'?** When a template is assigned, the milestone's definition and its rules are **copied** into these jsonb columns and frozen. If an admin later edits the library milestone, already-assigned shipments do **not** change. Always evaluate from the snapshot on the row — never re-read the library.

## 5. Resolving a field's value (do this everywhere)

Milestone checks reference CargoWise field names (e.g. `cargo_pickup_date`). To get the value for a given shipment, use the helper already in the codebase: `resolve_field_value(shipment, milestone_key, api_field)` in `services/field_registry.py`. It looks in three places, in order:

- Exact sub-key inside `shipments.milestones[milestone_key]`.
- A normalized/alias match inside that group (case/separator-insensitive, e.g. `First-Transit == first_transit_date`).
- Fallback to a plain shipment column: `shipment.get(api_field)` — this is how manually-maintained columns resolve.

The `milestone_key` is in the snapshot. The `shipments.milestones` jsonb looks like this (real example from shipment S00863370):

```
"milestones": {
  "cargo_ready": { "cargo_ready_date": "2025-08-27T00:00:00" },
  "cargo_pickup": { "cargo_pickup_date": null,
  "pickup_date_status": "Delayed" }
}
```

So `resolve_field_value(shipment, 'cargo_pickup', 'cargo_pickup_date')` returns **null** (not picked up yet), and `resolve_field_value(shipment, 'cargo_pickup', 'pickup_date_status')` returns **"Delayed"**.

## 6. The milestone snapshot (config you evaluate)

`milestone_snapshot` is a flat JSON object with these keys (only the ones relevant to the type are filled):

| Key                                                                                   | Used by / meaning                                 |
|---------------------------------------------------------------------------------------|---------------------------------------------------|
| <code>milestone_type</code>                                                           | which check this is                               |
| <code>milestone_key</code>                                                            | registry key for <code>resolve_field_value</code> |
| <code>primary_field</code>                                                            | date / missing: the field being watched           |
| <code>field_a`, `operator`, `field_b`, `fixed_value`, `thresh<br/>old_value</code>    | comparison / status                               |
| <code>document_name`, `tracking_field</code>                                          | document                                          |
| <code>expected_date_source`, `expected_date_field`, `e<br/>xpected_date_offset</code> | how the due date is derived (section 8)           |
| <code>extra_logics`, `logic_combine</code>                                            | additional checks + AND/OR (section 7)            |
| <code>is_critical`, `name`, `description</code>                                       | display / escalation                              |

## 7. Evaluating one check (the 6 types)

Every milestone reduces to a boolean: **satisfied** (done) or **outstanding** (should alert). The primary check uses `milestone_type` + the fields above:

| Type    | Fields                                          | OUTSTANDING (should alert) when                             |
|---------|-------------------------------------------------|-------------------------------------------------------------|
| date    | <code>primary_field</code>                      | the field is null/empty (date not recorded yet)             |
| missing | <code>primary_field</code>                      | the field is null/empty                                     |
| status  | <code>field_a`, `operator`, `fixed_value</code> | the status match holds — evaluate identically to comparison |

| Type       | Fields                                           | OUTSTANDING (should alert) when                                                      |
|------------|--------------------------------------------------|--------------------------------------------------------------------------------------|
| comparison | field_a`, `operator`, `field_b` or `fixed_value` | the comparison is true                                                               |
| document   | tracking_field                                   | tracking_field` is null/empty (doc not uploaded).<br>`document_name` is a label only |
| custom     | (none)                                           | evaluate extra_logics only (section 7.2)                                             |

## 7.1 Two rules that save you work:

- **status IS comparison.** A status milestone is stored as field\_a operator fixed\_value where operator is 'equals' or 'not\_equals'. Implement comparison, and status is free.
- **document checks tracking\_field, not primary\_field.** This is the one place the watched field lives in a different key.

Comparison operators depend on Field A's data type:

| Field A type  | Operators (meaning)                                                                                                                         |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| date          | missing` (null); `more_than_x_days_before`/`more_than_x_days_after` (gap vs field_b/value > threshold_value days); `same_as`; `not_same_as` |
| number        | greater_than`, `less_than`, `equal_to`, `not_equal_to` (vs field_b/value; threshold_value = the number)                                     |
| text / status | equals`, `not_equals`, `contains`, `missing`                                                                                                |

The right-hand side is field\_b (resolve another field) OR fixed\_value (a literal string) — never both.

## 7.2 Multi-logic milestones (extra\_logics + logic\_combine):

A milestone may bundle extra checks in extra\_logics — a JSON array of blocks, each { type, ...fields } using the same field names as above. logic\_combine is 'and' or 'or'.

```
"extra_logics": [
{ "type":"status","field_a":"pickup_date_status",
"operator":"equals","fixed_value":"Completed" },
{ "type":"document","document_name":"Bill of Lading",
"tracking_field":"job_docs_last_edit_time" }
],
"logic_combine": "and"
```

The milestone's combined result = the primary check combined with every extra block via logic\_combine: **and** = all satisfied; **or** = any satisfied. For the custom type there is **no primary check** — evaluate the extra\_logics blocks alone (there is always at least one). Fire conditions that gate on 'still outstanding' (section 10) use this combined result.

## 8. Computing the due date

`expected_date_source` sets the deadline rule timing is measured against. The assignment step (`_compute_due_date` in `routes/templates.py`) already resolves most of these into `shipment_milestones.due_date` at assignment time. The exception is the last row — you resolve it at runtime.

| expected_date_source     | Due date                                                                                 | Who resolves    |
|--------------------------|------------------------------------------------------------------------------------------|-----------------|
| another_field            | value of <code>`expected_date_field`</code> + <code>`expected_date_offset`</code> days   | assignment      |
| days_after_creation      | shipment <code>`created_at`</code> + <code>`expected_date_offset`</code> days            | assignment      |
| after_previous_milestone | previous milestone's <code>`completed_date`</code> + <code>`expected_date_offset`</code> | YOU, at runtime |
| manual                   | set by admin at assignment                                                               | assignment      |

**after\_previous\_milestone (your job).** This is left null at assignment because the previous milestone hasn't finished yet. When a milestone at `sequence_order = N` becomes **completed**, find the milestone at `N+1` on the same shipment and set its `due_date = previous.completed_date + expected_date_offset` days. This is how a chained/sequential timeline works. A milestone with no due date can only run 'on\_date'/absolute rules — 'before'/'after' has no anchor.

## 9. Status values you maintain

Keep `shipment_milestones.status` current; the dashboard reads it:

| status    | Set it when                                                          |
|-----------|----------------------------------------------------------------------|
| pending   | assigned, due date in the future (or no due date), not yet satisfied |
| current   | due date is near / active window (optional, if you use it)           |
| overdue   | due date has passed and the milestone is still outstanding           |
| completed | the milestone's combined check is satisfied                          |

## 10. Alert rules (alert\_rules\_snapshot)

`alert_rules_snapshot` is an array; each rule fires independently. Fields per rule:

| Field               | Values / meaning                                                                                                                                           |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| timing              | 'before'   'on_date'   'after' — relative to <code>due_date</code> , using <code>`days_offset`</code> and <code>`fire_time`</code> (HH:MM local)           |
| condition           | 'always'   'if_not_recorded'   'if_comparison_true'   'if_missing' (see 10.1)                                                                              |
| recurrence_type     | 'once'   'daily'   'weekly'   'custom_interval' (every <code>`recurrence_interval`</code> days)                                                            |
| recurrence_end_type | 'after_n_times'( <code>`recurrence_end_n`</code> )   'on_date'( <code>`recurrence_end_date`</code> )   'when_condition_met'   'never'                      |
| stop condition      | <code>stop_condition_field`</code> + <code>`stop_condition_type`</code> ('is_not_null' 'is_null' 'changed' 'equals') + <code>`stop_condition_value`</code> |
| recipient_type      | 'operations'   'sales'   'consignee'   'custom' ( <code>`custom_email`</code> )                                                                            |

### 10.1 Fire conditions — a rule only sends if its condition is true at fire time:

| condition          | Fires only if                                                                    |
|--------------------|----------------------------------------------------------------------------------|
| always             | unconditionally at the scheduled time (a pure reminder / escalation)             |
| if_not_recorded    | the milestone's watched field is still empty (combined result still outstanding) |
| if_comparison_true | the comparison / status match is still true                                      |
| if_missing         | the required field is still empty                                                |

For multi-logic milestones, the non-'always' conditions test the **combined** outstanding result (section 7.2).

**10.2 Recipient resolution:** operations = the operations user assigned to the shipment; sales = the sales rep on the shipment (sales\_user\_email); consignee = consignee\_email; custom = the rule's custom\_email. Use services/email\_service.send\_email(to, subject, body).

## 11. The evaluation loop (pseudocode)

```

for m in load_active_shipment_milestones(): # status != 'completed'
    shipment = load_shipment(m.shipment_id)
    cfg = m.milestone_snapshot
    rules = m.alert_rules_snapshot

    satisfied = evaluate_combined(cfg, shipment) # sections 7 + 7.2
    if satisfied:
        set_status(m, 'completed', completed_date=now)
        resolve_next_due_date(m) # section 8 (after_previous)
        continue

    if m.due_date and now > m.due_date:
        set_status(m, 'overdue')

    for r in rules:
        if due_now(r, m.due_date) and condition_true(r, cfg, shipment, satisfied):
            if not stopped(r, m) and within_recurrence(r, m):
                send_email(recipient(r, shipment), subject(m,r), body(m,r))
                record_fire(r, m) # for recurrence + stop tracking

```

## 12. Worked example (real shipment)

Shipment **S00863370** (NEWTIMES, SEA). Its milestones jsonb: `cargo_ready.cargo_ready_date = 2025-08-27`, `cargo_pickup.cargo_pickup_date = null`, `pickup_date_status = "Delayed"`.

Assume a template assigned these two milestones:

- **Cargo Ready** — type date, primary\_field `cargo_ready_date`. Resolve → '2025-08-27' (not empty) → **satisfied** → status completed. If it is sequence\_order 0 and the next milestone is 'after\_previous\_milestone +2d', you set the next due\_date = 2025-08-27 + 2 = 2025-08-29.
- **Cargo Pickup Confirmation** — type custom, extra\_logics = [ date cargo\_pickup\_date filled AND status pickup\_date\_status=='Completed' ], combine 'and'. Resolve: cargo\_pickup\_date = null (block 1 outstanding) AND status = 'Delayed' != 'Completed' (block 2 outstanding) → combined **outstanding**. due\_date 2025-08-29. Today > due\_date → status overdue.
- Its rule (1 day before, if\_not\_recorded, daily, operations): condition true because still outstanding → fire an email to the shipment's operations user each day until cargo\_pickup\_date is filled or the status becomes 'Completed'.

## 13. Where things live in the code

| What                                       | File                                                         |
|--------------------------------------------|--------------------------------------------------------------|
| Field value resolution                     | services/field_registry.py` -> resolve_field_value()         |
| Due-date compute (assignment)              | routes/templates.py` -> _compute_due_date(), _snapshot_row() |
| Snapshot config keys list                  | routes/templates.py` -> _CONFIG_KEYS                         |
| What the sync writes into milestones jsonb | services/cargowise_service.py` -> build_milestones()         |
| Registry (which fields the sync collects)  | milestone_field_map table; `routes/field_map.py              |
| Sending email                              | services/email_service.py` -> send_email()                   |
| Scheduler pattern to copy                  | app.py` (APScheduler jobs)                                   |

## 14. Prerequisites & guarantees

- Migrations that must be applied: `milestone_multi_logic.sql` (adds extra\_logics / logic\_combine columns).
- The sync already collects every field a milestone references — including fields inside extra\_logics — into `shipments.milestones`. You do not need to touch the sync or the registry.
- You only read `shipment_milestones` + the shipment. You write only: `shipment_milestones.status`, `completed_date`, and the runtime `due_date` for `after_previous_milestone` rows; plus your own alert-fire log for recurrence/stop tracking.
- There is no per-block AND/OR grouping and no stage-gating within one milestone — combine is milestone-wide (all blocks one way).